



DOCUMENTO TÉCNICO DE REFERENCIA

Infraestructura *técnica*

Tomando el Control

Todo el stack que sostiene el sistema: dominio, hosting, código, base de datos, automatización de emails, calendario y tracking. Pensado para que cualquier persona técnica futura (o tú con un asistente) pueda mantener, modificar o extender el sistema sin perder el contexto.

CONTENIDO

¿Qué encontrarás *en este documento?*

Una descripción completa de cada pieza del stack, cómo se conectan entre sí y cómo modificar cada una sin romper las otras.

01	Arquitectura general • El stack en una mirada	3
02	Repositorio GitHub • Código fuente	4
03	Netlify • Hosting + HTTPS + redirects	5
04	Apps Script • El cerebro automatizado	6
05	Google Sheets • La base de datos de leads	8
06	Google Drive • Los reportes PDF	9
07	Calendly • Agenda de llamadas	9
08	Endpoints públicos • URLs que sirve el sitio	10
09	Workflow de deploy • Cómo subir cambios	11

01 • ARQUITECTURA GENERAL

El stack *en una mirada*

7 servicios externos hablando entre sí. Todo está en planes gratuitos (excepto Google Workspace que ya pagabas).

① Servicios y costos

SERVICIO	FUNCIÓN	COSTO	LÍMITE RELEVANTE
GoDaddy	Dominio + DNS	~\$15/año	—
Netlify	Hosting + HTTPS + redirects	Gratis	100 GB bandwidth, 300 build min/mes
GitHub	Código fuente + auto-deploy	Gratis	Ilimitado para repos públicos
Google Sheets	Base de datos de leads	Gratis	10 M celdas por hoja
Apps Script	API web + envío de emails + triggers	Gratis	20 K llamadas/día, 100 emails/día Gmail
Google Drive	Almacenamiento de los PDFs de reporte	Incluido en Workspace	30 GB
Gmail (Workspace)	Envío de emails con dominio propio	\$7/mes	100 emails/día por usuario
Calendly	Agenda de llamadas de 30 min	Gratis	1 event type

② Cómo se conectan

tupmondemand.com

Frontend público (landing + quiz). Servido por Netlify desde el repo de GitHub.

/wa, /pausar

Páginas estáticas que hacen POST al Apps Script en background.

Apps Script

Recibe POSTs del quiz, escribe al Sheet, manda emails desde Gmail, lee PDFs de Drive.

Sheet "Quiz Leads"

Toda la data persistida. 21 columnas. Single source of truth.

Drive "Perfiles"

6 PDFs (uno por perfil). Apps Script los adjunta en cada E0.

Calendly

Linkeado en los emails E0 y E1. Sincroniza con tu Google Calendar.

02 • REPOSITORIO GITHUB

El código *fuentes*

Todo el código del quiz + landing + Apps Script vive en un solo repo público.

① Datos clave

URL	<code>github.com/analiapuello0209/quiz-pmondemand</code>
Visibilidad	Público
Branch principal	<code>main</code>
Framework del quiz	Create React App (CRA) – no Vite
Deploy automático	Sí, cada push a <code>main</code> dispara Netlify

② Estructura de carpetas

```
quiz-pmondemand/
├── public/
│   ├── index.html      ← entry del React (se renombra a quiz.html en build)
│   ├── landing.html    ← landing autocontenido, 723 KB
│   ├── pausar.html     ← página de unsubscribe
│   ├── wa.html         ← redirect a WhatsApp con tracking
│   └── perfiles/        ← 6 PNGs del abanico (usados en quiz y landing)
├── src/
│   ├── App.jsx         ← TODO el quiz (un componente, ~970 líneas)
│   └── index.js        ← entry point React
├── apps-script/        ← código del Apps Script (versionado con clasp)
│   ├── Código.js       ← lógica principal del backend
│   ├── appsscript.json  ← manifest
│   ├── .clasp.json     ← config local
│   └── email_e0/e1/e2_*.html ← 18 templates HTML
├── docs/               ← este documento + customer journey
├── netlify.toml        ← config del hosting (CRÍTICO, ver pág. 5)
└── package.json
```

Nunca renombres `public/index.html` : CRA lo necesita con ese nombre exacto para compilar. El renombrado a `quiz.html` ocurre en el build command de Netlify.

03 • NETLIFY

Hosting, HTTPS *y redirects*

Netlify hace 3 cosas: compila el React, sirve los archivos estáticos, y aplica las reglas de redirect del `netlify.toml`.

① Datos clave

Site interno	<code>brilliant-lolly-9cca77.netlify.app</code>
Dominio custom	<code>tupmondemand.com</code>
HTTPS	Automático (Let's Encrypt)
DNS	Configurado en GoDaddy: A record @ → 75.2.60.5, CNAME <code>www</code> → netlify
Deploys	Automático desde <code>main</code>

② El archivo netlify.toml

Este archivo es la lógica de routing del sitio. Si lo borras, todo se rompe.

```
[build]
  command = "npm run build && mv build/index.html build/quiz.html"
  publish = "build"

[[redirects]]
  from = "/quiz"
  to = "/quiz.html"
  status = 200

[[redirects]]
  from = "/pausar"
  to = "/pausar.html"
  status = 200

[[redirects]]
  from = "/wa"
  to = "/wa.html"
  status = 200

[[redirects]]
```

```
    from = "/teaser"  
    to = "/quiz"  
    status = 301  
  
[[redirects]]  
    from = "/*"  
    to = "/landing.html"  
    status = 200
```

⚠ EL TRUCO QUE HACE QUE TODO FUNCIONE

El `mv build/index.html build/quiz.html` en el build command es lo que permite que la landing sea homepage y el quiz viva en `/quiz`. Sin este renombrado, React serviría siempre desde `/` y la landing sería inaccesible.

04 • APPS SCRIPT

El cerebro *automatizado*

Apps Script es lo que orquesta todo: recibe los datos del quiz, los guarda, manda los emails, registra clicks, suprime envíos si la persona pausa o ya conversó por WhatsApp.

① Datos clave

Proyecto	Quiz Leads API
Cuenta	analia@tupmondemand.com
Script ID	1q_BENzHkKs_s6_HmX1pSsVM3g3hYukjsyZnlmBsRIzrtpbAnfTazUGFi
Web App URL	script.google.com/macros/s/AKfycbxwEM5yy6EGHCP0nu1_UAM18f_0hg9MD0vXckxwuUqsaxhHFIsudwzbqCVy1GtUeSfj/exec
Acceso	Cualquiera (necesario para que el quiz le pueda hacer POST sin auth)
Trigger	Time-based, diario 9-10 AM hora Panamá → dailyEmailJob

② Funciones principales

FUNCIÓN	QUIÉN LA DISPARA	QUÉ HACE
doPost(e)	POST desde quiz/pausar/wa	Router: lee data.type y llama el handler correcto
handleQuizResult	quiz completado	Inserta fila nueva con respuestas + perfil
handleEmailCapture	click "Quiero mi reporte"	Guarda email + código + dispara E0
handleUnsubscribe	click "puedes pausar aquí"	Marca col 20 unsubscribed_at

FUNCIÓN	QUIÉN LA DISPARA	QUÉ HACE
<code>handleWhatsAppClick</code>	click botón WA en email	Marca col 21 wa_clicked_at
<code>dailyEmailJob</code>	trigger diario 9 AM Panamá	Manda E1 (día +3) y E2 (día +6) con supresión
<code>sendEmailE0/E1/E2</code>	las anteriores	Carga template, interpola variables, manda con PDF adjunto (solo E0)

04 • APPS SCRIPT (CONT.)

Constantes *y* templates

③ Constantes configurables

Si necesitas cambiar algo "global" (timezone, URLs, sender), está acá:

```
const SENDER_NAME      = "Analía Puello · PM On Demand";
const BASE_URL          = "https://tupmondemand.com";
const CALENDLY_URL      = "https://calendly.com/analía-tupmondemand/30min";
const PDF_FOLDER_ID     = "1m30Qe9hv2baj5xBENzzD3romrG9exNqX";
const DAYS_BETWEEN_E0_E1 = 3;
const DAYS_BETWEEN_E1_E2 = 3;

const SUBJECTS = {
  "e0": "tu resultado del quiz",
  "e1": "una pregunta",
  "e2": "antes de que cierre tu código",
};
```

④ 18 templates HTML

Cada perfil tiene 3 emails (E0 + E1 + E2). Total: 18 archivos HTML separados en el Apps Script editor:

STAGE	CUÁNDO SE MANDA	PDF ADJUNTO	TONO
E0	Inmediato al capturar email	Sí, el reporte completo del perfil	Validación + oferta + descuento
E1	Día +3	No	Conversacional, una pregunta abierta
E2	Día +6	No	Urgencia suave (deadline del código)

⑤ Versionado con clasp

El código se mantiene en local (carpeta `apps-script/`) y se sube al editor de Apps Script con `clasp`. Esto evita el copy-paste manual.

```
# Sincronizar local → remoto  
cd apps-script  
clasp push --force
```

```
# Crear nueva versión del web app (mantiene la URL)  
clasp deploy -i AKfycbxw...GtUeSfj -d "descripción del cambio"
```

✓ BENEFICIO

Cualquier asistente técnico futuro puede editar los `.html` o el `Código.js` en local, correr 2 comandos, y el cambio queda live en producción sin tocar el editor web de Google.

05 • GOOGLE SHEETS

La base *de datos*

Una sola hoja con 21 columnas. Cada fila = una persona que hizo el quiz. Apps Script lee y escribe acá.

① Acceso

Nombre Quiz Leads

URL `docs.google.com/spreadsheets/d/17bSj5m2fKFEEduqRiNgWB6BokCBfZcbg_zS5dL7VTMs/`

Dueño analia@tupmondemand.com

② Schema de las 21 columnas

#	NOMBRE	QUIÉN LA ESCRIBE	CONTENIDO
1	timestamp	quiz	ISO timestamp del submit
2	type	quiz	"quiz_result"
3	name	quiz	nombre completo
4	role	quiz	rol del step 2
5	answers	quiz	respuestas P1-P11 comma-separated
6	goal	quiz	respuesta abierta P12
7	profile_primary	quiz	perfil principal calculado
8	profile_secondary	quiz	perfil secundario o N/A
9	monthly_cost	quiz	costo del caos en USD/mes
10	hrs_lost_day	quiz	horas no productivas/día
11	hourly_rate	quiz	tarifa por hora de P8

#	NOMBRE	QUIÉN LA ESCRIBE	CONTENIDO
12	email	quiz + email_capture	email (puede venir del step 1 o del OfertaFinal)
13	code	email_capture	código TC-XXX-NNNN
14	age_range	quiz	18-24, 25-34, 35-44, 45-54, 55+
15	code_expires	email_capture	YYYY-MM-DD (hoy + 7 días)
16	unsubscribe_token	email_capture	UUID único para link de pausar
17	email_e0_sent_at	sendEmailE0 OK	timestamp ISO
18	email_e1_sent_at	dailyEmailJob	timestamp ISO o vacío
19	email_e2_sent_at	dailyEmailJob	timestamp ISO o vacío
20	unsubscribed_at	handleUnsubscribe	timestamp ISO o vacío
21	wa_clicked_at	handleWhatsAppClick	timestamp ISO o vacío

06 • GOOGLE DRIVE • 07 • CALENDLY

PDFs *y* calendario

① Carpeta de PDFs en Drive

Apps Script accede a Drive para adjuntar el PDF correcto en cada E0.

Folder ID `1m30Qe9hv2baj5xBENzzD3romrG9exNqX`

URL `drive.google.com/drive/folders/1m30Qe9hv2baj5xBENzzD3romrG9exNqX`

Archivos en la carpeta (orden importa)

- ✓ **01_Reporte Apagafuegos.pdf** — 414 KB
- ✓ **02_Reporte Todoterreno.pdf** — 537 KB
- ✓ **03_Reporte Perfeccionista.pdf** — 427 KB
- ✓ **04_Reporte Multitasker.pdf** — 423 KB
- ✓ **05_Reporte Planificador.pdf** — 548 KB
- ✓ **06_Reporte Optimizador.pdf** — 541 KB

⚠ SI ACTUALIZAS UN PDF

Mantén exactamente el mismo nombre de archivo. Apps Script los busca por nombre en el map `PROFILE_PDF_MAP`. Si renombas, hay que actualizar el código.

② Calendly

URL pública `calendly.com/analía-tupmondemand/30min`

Event type Conversación • Tomando el Control (30 min)

Integración Sincronizado con tu Google Calendar

Dónde se usa Variable `{{calendly_url}}` interpolada en emails E0 y E1

Si cambias la URL del Calendly (porque cambias de plan, etc.), solo necesitas actualizar la constante `CALENDLY_URL` en `Código.js` y re-deployar con clasp. Los emails ya enviados conservan la URL vieja.

08 • ENDPOINTS PÚBLICOS

URLs que *sirve el sitio*

Todo el funnel pasa por 5 URLs públicas. Aquí están con explicación de qué hace cada una.

URL	QUÉ SIRVE	QUIÉN LA CONSUME
<code>tupmondemand.com</code>	Landing page completa	Visitante orgánico, redes sociales, etc.
<code>tupmondemand.com/quiz</code>	El quiz React (todo el flujo)	Click desde la landing
<code>tupmondemand.com/pausar?token=X</code>	Página de unsubscribe	Click en "puedes pausar aquí" de emails
<code>tupmondemand.com/wa?code=X&profile=Y&from=Z</code>	Tracker + redirect a WhatsApp con mensaje pre-armado	Click en botón WhatsApp de emails E0/E2
<code>tupmondemand.com/teaser</code>	Redirect 301 permanente a <code>/quiz</code>	Links viejos guardados de antes que se reemplazó el teaser por el quiz

① Cómo funcionan las páginas estáticas con backend

`/pausar` y `/wa` son páginas HTML estáticas servidas por Netlify, pero hacen **POST en background** al Apps Script para registrar la acción. El usuario ve la página inmediata (sin esperar respuesta del servidor) y el tracking ocurre asíncrono.

↳ POR QUÉ PÁGINAS ESTÁTICAS Y NO PROXY

Probamos primero un proxy directo de Netlify al Apps Script, pero el plan free de Netlify restringe rewrites a URLs externas. La solución estática + JS background es más robusta, más rápida (no espera al servidor), y la URL queda limpia (`tupmondemand.com/pausar`).

09 • WORKFLOW DE DEPLOY

Cómo subir *cambios*

Hay 2 sistemas que deployan distinto: el sitio web (Netlify) y el backend (Apps Script).

① Cambios al sitio web (quiz / landing / páginas estáticas)

1. Editar archivos en `src/` o `public/`
2. Commit + push a `main`
3. Netlify detecta el push, builds, deploys (~2 min)
4. Cambio live en `tupmondemand.com`

② Cambios al backend (Apps Script)

1. Editar archivos en `apps-script/` (Código.js o templates HTML)
2. Commit + push a GitHub (para tener historia)
3. Subir al editor remoto: `clasp push --force`
4. Crear nueva versión del web app: `clasp deploy -i AKfycbxw...GtUeSfj -d "descripción"`
5. Cambio live (la URL del web app queda igual, no toca App.jsx)

③ Verificaciones post-deploy

- ✓ **Sitio:** abrir `tupmondemand.com` en incógnita, completar el quiz con email de prueba
- ✓ **Sheet:** verificar que apareció una fila nueva con todos los campos llenos
- ✓ **Emails:** verificar que llegó E0 al inbox del email de prueba
- ✓ **Tracking:** click el botón WhatsApp → verificar que se llenó col 21
- ✓ **Pausar:** click "puedes pausar aquí" → verificar que se llenó col 20

BANDERA VERDE

Si las 5 verificaciones pasan, el deploy fue exitoso y la secuencia completa va a funcionar. El daily job se encargará de mandar E1 y E2 los días correspondientes sin intervención manual.